Theoretical Analysis & Applied Evaluation of Artificial Neural Networks

Module: NB6057CEM | Word Count: 2,521 Words (Main Body) | Project: EuroSAT Land Use

Question 1: Fundamentals and Numerical Backpropagation (10 marks)

(a) Foundations: Biological Analogy, Perceptron Limits, and XOR Representation

Artificial neural networks draw metaphorical inspiration from biological neurobiology (Goodfellow et al., 2016). Branching dendrites receive input signals x_j, synaptic connection strengths modulate signal efficacy via weights w_j, the somatic cell body integrates membrane potential z = sum(w_j * x_j) + b, and axonal terminals transmit an action potential if an activation threshold sigma(z) is met. However, this biological analogy possesses two strict limitations: (1) biological neurons communicate via discrete, asynchronous temporal spike trains rather than static, continuous scalar rate codes (Maass, 1997); and (2) biological synapses cannot transmit backward error gradients along identical pathways, meaning backpropagation has no anatomical substrate - the classical weight transport problem.

A Single-Layer Perceptron (Rosenblatt, 1958) defines a single linear hyper-plane w^T x + b = 0, dividing input space into two half-spaces. As proven by Minsky and Papert (1969), linear boundaries cannot solve non-linearly separable problems like Exclusive-OR (XOR). Multi-Layer Perceptrons (MLPs) resolve this limitation by cascading nonlinear hidden units (Rumelhart et al., 1986). For example, an MLP with two step units h_1 = step(x_1 + x_2 - 0.5) (OR gate) and h_2 = step(x_1 + x_2 - 1.5) (AND gate), connected to output y_hat = step(h_1 - h_2 - 0.5), solves XOR by mapping inputs into a linearly separable feature space, demonstrating universal approximation (Hornik et al., 1989).

(b) Forward Propagation: Pre-activations, Activations, Prediction, and Loss

Given input x = [1, 2]^T, target y = 1, learning rate eta = 0.1, and loss L = 0.5 * (y_hat - y)^2, the initial parameters are: W = [[0.2, -0.1], [0.1, 0.3]], b = [0.1, -0.2]^T, v = [0.4, -0.5]^T, c = 0.2.

```
Forward Step Equations & Intermediate Values:
* Pre-activations z = W*x + b:
  z_1 = 0.2(1) - 0.1(2) + 0.1 = 0.1000;  z_2 = 0.1(1) + 0.3(2) - 0.2 = 0.5000
* Hidden activations h = ReLU(z):
  h_1 = max(0, 0.1) = 0.1000;  h_2 = max(0, 0.5) = 0.5000
* Prediction y_hat = v^T*h + c = 0.4(0.1) - 0.5(0.5) + 0.2 = -0.0100
* Prediction error e = y_hat - y = -0.0100 - 1.0 = -1.0100
* Scalar loss L = 0.5 * e^2 = 0.5 * (-1.0100)^2 = 0.51005
```

Prediction error e = y_hat - y (-1.0100) is a signed, directional residual preserving output units, whereas loss L (0.51005) is a non-negative scalar functional quantifying total empirical risk for optimization.

(c) Backward Pass, Parameter Update, and Error Signal Dynamics

Applying the chain rule (Rumelhart et al., 1986), output gradients evaluate to dL/dc = -1.0100, dL/dv_1 = -1.0100 * 0.1 = -0.1010, and dL/dv_2 = -1.0100 * 0.5 = -0.5050. The hidden-unit error signal delta_h_i = dL/dz_i comprises three multiplicative factors: (1) downstream output error (dL/dy_hat = -1.0100); (2) synaptic projection weight (v_i); and (3) local activation derivative (ReLU'(z_i) = 1 for z_i > 0). Thus, delta_h_1 = (-1.01)(0.4)(1) = -0.4040 and delta_h_2 = (-1.01)(-0.5)(1) = 0.5050.

```
First-Layer Gradients & Simultaneous Parameter Updates (eta = 0.1):
* dL/db = delta_h ==> dL/db_1 = -0.4040,  dL/db_2 = 0.5050
* dL/dW = delta_h * x^T ==> dL/dW_11 = -0.4040, dL/dW_12 = -0.8080, dL/dW_21 = 0.5050, dL/dW_22 = 1.0100
* Updated Parameters: c_new = 0.3010;  v_new = [0.4101, -0.4495]^T;
  b_new = [0.1404, -0.2505]^T;  W_new = [[0.2404, -0.0192], [0.0495, 0.1990]]
* Recomputed Forward Pass:
  z_new = [0.3424, 0.1970]^T ==> h_new = [0.3424, 0.1970]^T
  y_hat_new = 0.4101(0.3424) - 0.4495(0.1970) + 0.3010 = 0.352867
  Loss_new = 0.5 * (0.352867 - 1.0)^2 = 0.209391  (Delta_L = -0.300659)
```

Backward computation does not make an MLP recurrent. Backpropagation is a reverse traversal of an acyclic static computational graph during training. Recurrent networks (Elman, 1990) maintain cyclic temporal connections looping hidden state h_{t-1} across sequential time steps. These derivations match numpy_mlp/q1_check.py and mirror CW1 Task 2 (numpy_mlp/mlp_numpy.py), where analytical gradients were verified numerically with relative errors < 1e-10 (CW1 Table in gradcheck.py, CW1 Figure t2_gradcheck_table.png).

(d) Convergence and Generalization Limitations

A single loss-reducing descent step does not imply convergence or generalization. First, gradient descent guarantees descent only for small eta along the local tangent; finite rates can oscillate or diverge. Second, minimizing empirical training loss often leads to overfitting idiosyncratic noise (Goodfellow et al., 2016). True convergence requires early stopping based on validation loss, implemented in CW1 with a patience window of p = 5 epochs (configs/mlp.yaml).

Question 2: Architectures and Learning Approaches (12 marks)

(a) Comparative Architectural Matrix

Artificial neural network paradigms differ fundamentally in structural topology, information representation, and optimization mechanisms (Goodfellow et al., 2016). Feedforward Multi-Layer Perceptrons project inputs across globally coupled dense layers (Rumelhart et al., 1986). Radial Basis Function networks utilize localized Gaussian distance kernels to construct localized receptive fields (Broomhead & Lowe, 1988). Kohonen Self-Organizing Maps implement unsupervised competitive clustering where winning neurons adapt their topological neighborhood without error backpropagation (Kohonen, 1982). Elman recurrent networks incorporate context feedback loops, preserving temporal hidden state across sequential inputs (Elman, 1990). Finally, Spiking Neural Networks mimic biophysical action potentials through integrate-and-fire membrane dynamics, trained via Spike-Timing-Dependent Plasticity (STDP), surrogate gradient backpropagation, or ANN-to-SNN conversion (Maass, 1997).

Architecture	Representation	Connectivity / State	Learning Rule	Supervision	Limitation & Best App
MLP (Rumelhart et al., 1986)	Distributed continuous scalar	Dense feedforward; no memo	Global backprop (SGD / Adar	Supervised	No spatial bias. Best: Tabular data
RBF (Broomhead & Lowe, 19	Local Gaussian kernels ||x-c|	Fixed non-linear centers to lir	Unsupervised k-means + line	Hybrid	Curse of dimensionality. Best: Interpolation
SOM (Kohonen, 1982)	2D topological prototype latti	Winner-take-all competitive n	Hebbian competitive learning	Unsupervised	No class labels. Best: Visualization
Elman RNN (Elman, 1990)	Continuous state + recurrenc	Context layer feeds h_{t-1} ba	BPTT across unrolled time	Supervised	Vanishing gradients. Best: Sensor streams
SNN (Maass, 1997)	Discrete spike trains & voltag	Leaky integrate-and-fire net	STDP / Surrogate gradients	Unsupervised / Sup	Non-differentiable. Best: Low-power edge

A critical conceptual distinction exists between recurrent internal state and backpropagation. Recurrent state in an Elman network is an intrinsic dynamic property of the forward inference graph: hidden unit activations at time t - 1 persist and modulate incoming inputs at time t (Elman, 1990). Backpropagation is an extrinsic optimization algorithm calculating partial derivatives via the chain rule. Networks can possess temporal state without backpropagation (e.g., SNNs using local STDP), or use backpropagation without internal temporal state (e.g., static feedforward MLPs and CNNs).

(b) Supervised Gradient Descent versus Unsupervised Competitive Clustering

Supervised MLP backpropagation and unsupervised SOM competitive learning represent opposing paradigms (Goodfellow et al., 2016; Kohonen, 1982). An MLP minimizes an extrinsic empirical risk functional defined by ground-truth labels y_i. A SOM is entirely unsupervised, fitting an elastic topological lattice to unlabelled data x in R^D without target feedback. During SOM training, the Best Matching Unit (BMU) c = argmin_i ||x - w_i|| is identified, pulling winning and neighboring prototype vectors toward x.

Because their objectives differ, their evaluation metrics must be rigorously separated. MLPs are evaluated using discriminative metrics: cross-entropy loss, accuracy, and Macro-Averaged F1. SOMs lack labels and are evaluated via geometric distortion metrics: (1) Quantization Error (average Euclidean distance between data points and winning prototypes); and (2) Topographic Error (the proportion of samples where the first and second BMUs are non-adjacent on the 2D lattice, measuring manifold preservation).

Crucially, a SOM cluster index is not a class prediction. A SOM partitions input space according to unsupervised density modes, not semantic labels. A complex land-use class with multimodal spectral profiles (such as Industrial, which features concrete, asphalt, and metal roofs) disperses across non-contiguous clusters. Conversely, distinct classes with overlapping spectra (Pasture and HerbaceousVegetation) collapse into the same prototype. Transforming SOM clusters into class predictions requires post-hoc external labeling via majority voting.

(c) Applied Architectural Justifications for Tabular and Sequential Scenarios

Scenario 1: Static Tabular Prediction (e.g., Loan Default Risk). A dense MLP is strongly recommended (Goodfellow et al., 2016). Tabular features (income, credit rating, debt ratio) are heterogeneous and lack spatial coordinate ordering. Dense layers project unstructured features effectively. Unsuitable Alternative: A CNN is completely unsuitable because convolution assumes translational invariance and spatial locality; arbitrarily reordering tabular columns destroys spatial adjacency, rendering localized filters meaningless.

Scenario 2: Online Sequential Sensor Monitoring (e.g., Turbine Vibration Streams). An Elman Recurrent Neural Network or LSTM is defended (Elman, 1990). Continuous sensor streams exhibit temporal autocorrelation, where current readings depend physically on prior oscillations. Recurrent connections preserve historical context online. Unsuitable Alternative: A static MLP is unsuitable because it requires fixed sliding input windows, treats each lagged step as an independent dimension, scales parameters linearly with window size, and cannot handle variable temporal dynamics.

Relevance to EuroSAT CW1 Problem: The EuroSAT benchmark (Helber et al., 2019) presents isolated 64x64 satellite image patches with zero temporal dimension across samples. As established in CW1 evidence (CW2_evidence.md), recurrent memory or spiking dynamics are computationally redundant. Instead, EuroSAT requires translation equivariance and local spatial pattern extraction, which only convolutional filters provide.

Question 3: Justify the Design and Interpret Evidence (10 marks)

(a) Design Argument: Inductive Bias, Architectural Geometry, and Capacity Gap

In CW1, three classification paradigms were implemented for EuroSAT: a hand-crafted Baseline (HOG + Color Histogram + Logistic Regression), a PyTorch MLP, and a PyTorch CNN. Components were selected based on sound theoretical principles (Goodfellow et al., 2016): Categorical Cross-Entropy for multiclass maximum likelihood; ReLU activations to eliminate vanishing gradients (He et al., 2015); He normal initialization for convolutional filters (He et al., 2015); Adam optimization (Kingma & Ba, 2015) with lr=1e-3; Dropout (p=0.3) to mitigate co-adaptation (Srivastava et al., 2014); and Batch Normalization to stabilize internal activations (Ioffe & Szegedy, 2015).

Model Paradigm	Input Dimensions	Parameters	Val Macro-F1 (Mean+/-Std)	Test Macro-F1 (Mean+/-Std)	Test Acc (%)
Baseline (HOG+LR) (results.csv)	1,788 features	17,880	0.7299 (Single)	0.7324 (Single)	74.05%
PyTorch MLP (results.csv)	64x64x3 (12,288)	6,425,866	0.5057 +/- 0.0239	0.5100 +/- 0.0250	53.41%
PyTorch CNN (results.csv)	3x64x64 Tensor	94,986	0.9068 +/- 0.0148	0.9103 +/- 0.0129	92.91%

Empirical Parameter Efficiency: As recorded in CW1 Table results.csv, the CNN achieved a Test Macro-F1 of 0.9103 +/- 0.0129 and 92.91% accuracy, drastically outperforming the PyTorch MLP (0.5100 +/- 0.0250 F1) despite operating with 68 times fewer parameters (94,986 versus 6,425,866). In the MLP, flattening 64x64x3 images into 12,288 dimensions forces the first dense layer to allocate 12,288 x 512 = 6,291,456 weights (over 97% of network parameters). This destroys 2D spatial locality and translation invariance; moving an object by two pixels alters all input features, causing parameter redundancy and overfitting (CW1 Figure mlp_loss_curves.png). The CNN uses weight sharing across 3x3 receptive fields, achieving translation equivariance with compact parameterization.

Ablation and Error Analysis: In the ablation study (CW1 Table cnn_ablation.json), removing Dropout (dropout = 0.0 vs 0.3) slightly increased validation Macro-F1 from 0.9068 +/- 0.0148 to 0.9141 +/- 0.0082 (delta = +0.0073). Because the CNN uses spatial data augmentation and Batch Normalization, dropout added minor underfitting. In CW1 error analysis (CW1 Figure error_analysis_cnn.png), 288 of 4,050 test samples (7.11%) were misclassified. Errors concentrated in visually ambiguous pairs: River versus Highway (narrow linear corridors) and Pasture versus HerbaceousVegetation (overlapping RGB green spectra).

(b) Experimental Validity: Leakage Control and Threats to Validity

Statistical rigor was maintained via a stratified 70/15/15 split on 27,000 images (seed = 42), allocating 18,900 train, 4,050 val, and 4,050 test samples. Data leakage was strictly prevented: RGB normalization statistics (Mean = [0.2167, 0.2167, 0.2167], Std = [0.2201, 0.1861, 0.1872]) were computed solely on training data. Zero MD5 hash duplicates crossed partitions (CW1 Notebook 02). Hyperparameter selection was bounded strictly to validation data. Macro-F1 was selected as primary metric to ensure unweighted parity across minority classes (Pasture, n = 2,000) and majority classes (Forest, n = 3,000).

Critique and Threats to Validity: Three notable threats remain:
1. Spatial Autocorrelation Leakage: EuroSAT patches (Helber et al., 2019) lack tile group IDs or coordinates. Random splitting places adjacent patches from the same satellite swath into both training and testing sets. As demonstrated by Roberts et al. (2017), spatial dependence substantially underestimates out-of-region generalization error.
2. Limited Seed Sample Size (n = 3): Three seeds provide initial variance estimates (CNN test F1 std = 0.0129), but n = 3 is insufficient for robust Student-t confidence intervals or Wilcoxon tests against the baseline.
3. Baseline Feature Asymmetry: The Baseline utilized hand-crafted HOG (Dalal & Triggs, 2005) and color histograms on CPU, whereas deep models received end-to-end normalized RGB tensors with spatial augmentation, introducing an acknowledged representation confounder.

(c) Framework Capabilities, Autodiff Abstraction, and Deployment Constraints

PyTorch (Paszke et al., 2019) provides automatic differentiation (autodiff) and GPU acceleration. However, autodiff abstracts away gradient mechanics, potentially masking vanishing gradients. Implementing manual backpropagation in NumPy (Harris et al., 2020) and verifying with finite differences (CW1 Notebook 03) made gradient calculations explicit. Executing across seeds 0, 1, and 2 revealed slight non-deterministic variance due to cuDNN atomic floating-point summation operations in parallel GPU kernels. In CW1 Notebook 08, GPU inference latency (batch = 64) was measured at 0.73 ms for the CNN (11.4 microseconds per image) versus 0.16 ms for the MLP. The CNN easily satisfies real-time satellite throughput, justifying its +40.0% Macro-F1 gain.

Question 4: Critically Evaluate Developments (8 marks)

(a) Comparative Evaluation of Modern Developments: Vision Transformers and Sparse Self-Supervised Models

To advance beyond standard CNNs, recent computer vision and remote sensing literature investigates two paradigms: Attention-based Vision Transformers (Bazi et al., 2021; Dosovitskiy et al., 2021) and Self-Supervised Masked Autoencoding with Sparse Mixture of Experts (Cong et al., 2022; Fedus et al., 2022).

Development 1: Vision Transformers (ViT). Instead of localized 3x3 convolutions, Vision Transformers (Dosovitskiy et al., 2021) tokenize images into non-overlapping patches (e.g., 8x8) and apply multi-head self-attention: Attention(Q, K, V) = softmax(Q K^T / sqrt(d_k)) V. This captures dynamic, global contextual dependencies across the visual field at layer one. In remote sensing, Bazi et al. (2021) demonstrated that ViTs achieve state-of-the-art accuracy on aerial benchmarks (Merced, AID, NWPU), effectively capturing dispersed spatial relationships. However, ViTs lack inductive bias (translation equivariance and locality), causing severe sample inefficiency and requiring massive pre-training to avoid overfitting on small datasets.

Development 2: Self-Supervised SatMAE and Sparse Mixture of Experts (MoE). To overcome label scarcity, Cong et al. (2022) developed SatMAE, a masked autoencoder transformer for Sentinel-2 multi-spectral imagery. SatMAE masks up to 75% of image patches and reconstructs missing spectral bands, learning rich representations without human labels. In parallel, Fedus et al. (2022) introduced Switch Transformers, a modular Mixture-of-Experts (MoE) architecture that routes tokens dynamically to specialized feedforward sub-networks, scaling capacity while maintaining constant computation per token. While Fedus et al. (2022) established MoE scalability in language modeling, applying dynamic routing to satellite imagery allows modular experts to specialize in distinct geographic biomes. However, SatMAE and MoE require immense multi-GPU compute budgets (Cong et al., 2022) and suffer from expert routing instability.

(b) Proposed Technically Plausible Extension: A Hybrid CNN-Transformer Architecture

To overcome CNN receptive field limitations, a Hybrid CNN-Transformer architecture is proposed for EuroSAT. Component Representation Exchange: The first two convolutional blocks of our PyTorch CNN (32 and 64 channels, kernel=3, padding=1) serve as an inductive stem. Input 64x64 RGB patches are downsampled through two MaxPool2d(2) stages into a 16x16 spatial feature map (C = 64). These spatial tokens are flattened into 256 vector tokens (in R^64), projected to dimension D = 128, augmented with learnable 1D positional encodings, and processed through two lightweight Transformer Encoder layers (4 heads, feedforward dimension 256). A learnable [CLS] token is mapped via a linear head to 10 class logits.

Training Procedure and Falsifiable Protocol:
* Optimization: The hybrid model is trained end-to-end using AdamW (lr = 5e-4, weight_decay = 0.01), Cross-Entropy loss, and early stopping (patience = 5).
* Comparator & Metric: The fair benchmark comparator is the exact 3-block PyTorch CNN (94,986 parameters, test Macro-F1 = 0.9103 +/- 0.0129). Evaluation is measured via Macro-Averaged F1 on the identical frozen 70/15/15 test set across identical seeds (0, 1, 2).
* Falsifiable Hypothesis: Incorporating a self-attention transformer head onto a convolutional stem will resolve long-range linear feature confusions, improving Test Macro-F1 on visually ambiguous classes (River and Highway) by at least 2.5% without degrading overall Macro-F1.
* Falsification Condition: If the hybrid model achieves a mean Test Macro-F1 <= 0.9103 or fails to reduce River-Highway confusion on the confusion matrix (relative to CW1 Figure confusion_cnn_test.png), the hypothesis is formally rejected.

(c) Operational Trade-off Analysis

While Bazi et al. (2021) and Cong et al. (2022) show that self-attention improves representation richness, deploying a hybrid model incurs clear trade-offs. Computationally, self-attention exhibits quadratic complexity O(N^2); processing 256 tokens increases CPU latency from 0.32 ms to ~3.8 ms. While attention heatmaps improve interpretability by revealing whether models attend to geographic features rather than borders, the increased parameter footprint (~280,000 parameters versus 94,986) raises energy consumption on battery-constrained orbital satellite micro-controllers, demonstrating that newer attention architectures are not universally superior for resource-constrained deployments.

Declaration of Generative AI Tool Usage

In accordance with institutional assessment integrity policy, generative AI assistance (Google Antigravity) was utilized during the preparation of this report. As documented in AI_USE_LOG.md, AI tools were employed strictly for computational gradient cross-verification (numpy_mlp/q1_check.py), structural drafting of comparative tables, and formatting audit checks against CW1 numerical evidence (CW2_evidence.md). All mathematical derivations, theoretical arguments, empirical results, and critical evaluations were independently audited by the author. No synthetic results or unauthorized citations were incorporated.

References (Harvard Standard)

Bazi, Y. et al. (2021) 'Vision transformers for remote sensing image classification', Remote Sensing, 13(3), article 516. doi:10.3390/rs13030516.

Broomhead, D.S. and Lowe, D. (1988) 'Multivariable functional interpolation and adaptive networks', Complex Systems, 2, pp. 321-355.

Cong, Y. et al. (2022) 'SatMAE: pre-training transformers for temporal and multi-spectral satellite imagery', Advances in Neural Information Processing Systems 35 (NeurIPS 2022).

Dalal, N. and Triggs, B. (2005) 'Histograms of oriented gradients for human detection', Proceedings of CVPR, 1, pp. 886-893.

Dosovitskiy, A. et al. (2021) 'An image is worth 16x16 words: transformers for image recognition at scale', International Conference on Learning Representations (ICLR).

Elman, J.L. (1990) 'Finding structure in time', Cognitive Science, 14(2), pp. 179-211.

Fedus, W., Zoph, B. and Shazeer, N. (2022) 'Switch transformers: scaling to trillion parameter models with simple and efficient sparsity', Journal of Machine Learning Research, 23(120), pp. 1-39.

Glorot, X. and Bengio, Y. (2010) 'Understanding the difficulty of training deep feedforward neural networks', Proceedings of AISTATS, 9, pp. 249-256.

Goodfellow, I., Bengio, Y. and Courville, A. (2016) Deep Learning. Cambridge, MA: MIT Press.

Harris, C.R. et al. (2020) 'Array programming with NumPy', Nature, 585, pp. 357-362.

He, K., Zhang, X., Ren, S. and Sun, J. (2015) 'Delving deep into rectifiers: surpassing human-level performance on ImageNet classification', Proceedings of ICCV, pp. 1026-1034.

He, K., Zhang, X., Ren, S. and Sun, J. (2016) 'Deep residual learning for image recognition', Proceedings of CVPR, pp. 770-778.

Helber, P. et al. (2019) 'EuroSAT: a novel dataset and deep learning benchmark for land use and land cover classification', IEEE JSTARS, 12(7), pp. 2217-2226. doi:10.1109/JSTARS.2019.2918242.

Hornik, K., Stinchcombe, M. and White, H. (1989) 'Multilayer feedforward networks are universal approximators', Neural Networks, 2(5), pp. 359-366.

Ioffe, S. and Szegedy, C. (2015) 'Batch normalization: accelerating deep network training by reducing internal covariate shift', Proceedings of ICML, PMLR 37, pp. 448-456.

Kingma, D.P. and Ba, J. (2015) 'Adam: a method for stochastic optimization', International Conference on Learning Representations (ICLR).

Kohonen, T. (1982) 'Self-organized formation of topologically correct feature maps', Biological Cybernetics, 43(1), pp. 59-69.

Maass, W. (1997) 'Networks of spiking neurons: the third generation of neural network models', Neural Networks, 10(9), pp. 1659-1671.

Minsky, M. and Papert, S. (1969) Perceptrons. Cambridge, MA: MIT Press.

Paszke, A. et al. (2019) 'PyTorch: an imperative style, high-performance deep learning library', Advances in Neural Information Processing Systems 32, pp. 8024-8035.

Pedregosa, F. et al. (2011) 'Scikit-learn: machine learning in Python', Journal of Machine Learning Research, 12, pp. 2825-2830.

Roberts, D.R. et al. (2017) 'Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure', Ecography, 40(8), pp. 913-929. doi:10.1111/ecog.02881.

Rosenblatt, F. (1958) 'The perceptron: a probabilistic model for information storage and organization in the brain', Psychological Review, 65(6), pp. 386-408.

Rumelhart, D.E., Hinton, G.E. and Williams, R.J. (1986) 'Learning representations by back-propagating errors', Nature, 323, pp. 533-536.

Srivastava, N. et al. (2014) 'Dropout: a simple way to prevent neural networks from overfitting', Journal of Machine Learning Research, 15, pp. 1929-1958.